

ATC Readback Verifier

Technical Report

A hybrid system that verifies a pilot read-back against a controller instruction — an LLM extracts structured fields, deterministic Python judges every discrepancy.

TEAM

Jan · Vlad · Felipe · Alberto · Kishan · Yi

REPOSITORY

github.com/janwejchert/NLP

DATE

June 2026

Proof of concept — not operational software.

ATC Readback Verifier — Technical Report

NLP Group Project (Option 1 — Application Development). A proof-of-concept verification system, not production or operational software. **Team (6):** Jan, Vlad, Felipe, Alberto, Kishan, Yi. **Repository:** <https://github.com/janwejchert/NLP> — Live demo on Streamlit Community Cloud.

Abstract

In air traffic control (ATC), a pilot reads a controller's instruction back so the controller can confirm it was heard correctly; an uncaught read-back error is a documented contributor to aviation incidents. We present the **ATC Readback Verifier**, a Streamlit web app that takes a controller instruction and a pilot read-back as text, extracts eight structured safety fields from each, and compares them with a deterministic Python comparator to return MATCH or a list of specific discrepancies. The architecture is deliberately hybrid: an LLM performs *only* extraction, while *all* correctness judgement is deterministic and auditable. On our own 50-case labelled test set, the final configuration (`qwen2.5:3b`) reaches F1 0.986 for error detection, a 6.2% false-alarm rate, and 100% detection recall in every error category. The most instructive result is a three-stage iteration showing that the comparator was never at fault — extraction is the bottleneck, and over-specific few-shot examples can *poison* a small model.

1. Introduction and justification of need

ATC communication relies on a closed loop: the controller issues a clearance, the pilot reads back the safety-relevant elements, and the controller listens to confirm correctness (the *hear-back*). This loop is the primary defence against miscommunication, yet it depends entirely on a human catching an error in real time, and an uncorrected error can remain undetected until a loss of separation appears on the controller's display. Foundational safety research quantifies the gap: FAA/Volpe analyses of NASA ASRS reports find that controllers catch only roughly two-thirds of incorrect read-backs, leaving about a third undetected, with error likelihood rising as a transmission carries more items.

Existing ATC speech and language tooling overwhelmingly targets a *different* task — **transcription**, turning radio audio into text — rather than **verification**, checking that a read-back faithfully matches the instruction. Commercial systems advertise transcription for cockpit displays but no clearance-verification function, and even the large public ATC corpora are built around collection, transcription, and concept extraction rather than a fielded verifier. Dedicated read-back-error-detection prototypes exist, almost entirely from European SESAR-funded research, but they remain at the lab stage with non-trivial false-alarm rates.

Our contribution is positioned squarely in that under-served niche: a lightweight, auditable, **text-based** field-by-field verifier with an explicit error taxonomy. We deliberately *verify correctness, not transcribe*. Audio / ASR is out of scope and treated as future work, which lets the project be a focused verification layer rather than another transcription front-end. As a course proof-of-concept we make no operational claims; the goal is a defensible, reproducible demonstration of the verification idea.

2. Field review (condensed synthesis)

Read-back/hear-back failure is a long-recognised aviation safety problem. SKYbrary and the ICAO information paper distinguish three failure modes — a read-back error the controller fails to catch, an absent read-back, and a "Type II" hear-back error where the controller misses a mistake in their *own*

clearance even when the read-back was correct [1][2]. NASA ASRS work coined the "hear-back problem" and an associated study analysed several hundred reports [3][4]; the FAA/Volpe analysis by Cardosi and colleagues reports that controllers catch only ~two-thirds of incorrect read-backs and that error likelihood rises with message complexity, with headings, frequency changes, and altitudes among the most error-prone items [5][6]. Consistent contributing factors include confusable callsigns, expectation bias (e.g. hearing heading 280 as flight level 280), workload, and non-standard phraseology [1][3]. ICAO Annex 10, PANS-ATM (Doc 4444), and the Manual of Radiotelephony (Doc 9432) define the mandatory read-back items — callsign, runway, level/altitude, heading, speed, transponder code, QNH — which directly supplies a verifier's ground-truth field set [7][8], and EUROCONTROL's AGC action plan reinforces strict read-back discipline [9].

On the technology side, the ATCO2 corpus frames ATC understanding around three entity classes (callsign, command, value) and reports a fine-tuned-BERT NER baseline (F1 ~0.97 callsign, ~0.82 command, ~0.87 value), with command extraction hardest because utterances chain multiple commands and deviate from published phraseology [10][11]; ATCOSIM provides a cleaner benchmark [12], and self-supervised and Whisper-based ASR models have been fine-tuned and released for ATC [13][14], with surveillance-context fusion improving callsign recognition [15]. The clearest gap is that mature systems transcribe rather than verify [16][17], while the closest verification work — Helmke et al. in the HAAWAI project — performs concept-level comparison of controller-instruction against pilot read-back and reports ~80–81% detection on real recordings, finding semantic-concept comparison more robust than word-level matching [18][19]. Our design mirrors that concept-comparison insight while staying transparent: the LLM produces the ICAO mandatory items as structured fields and every judgement is deterministic Python. **The full review, with all references and verified source links, is in `field_review.md`.**

3. System design

Hybrid pipeline. The system runs two extraction passes — one over the instruction, one over the read-back — that each turn free text into the same typed schema, then hands both to a deterministic comparator that emits a verdict:



The LLM does *only* structured extraction — the well-bounded part a small free model handles well. All judgement (what counts as a match, normalization, which error category applies) is deterministic Python the team wrote, which maximises originality, reproducibility, and defensibility.

Eight fields. Extraction produces a typed schema of `callsign`, `altitude`, `heading`, `speed`, `frequency`, `squawk`, `runway`, and `qnh`, each `None` when absent. These map onto the ICAO mandatory read-back items.

Normalization decisions. Comparison is done on normalized values so that semantically equal read-backs match: spelled-out numbers become digits ("three thousand" → 3000), telephony is normalized ("one zero one three" → 1013), flight levels and feet are handled distinctly (FL280 vs 3000 ft), frequencies have trailing-zero normalization, squawk leading zeros are preserved, and runway captures designator plus optional side. For example, "climb three thousand feet, QNH 1013" is treated as equal to "climb altitude 3000 feet, QNH 1013".

Deterministic comparator and the five-category error taxonomy. The comparator compares each instruction field against its read-back counterpart and emits zero or more `Discrepancy` records. A read-back with no discrepancies is a MATCH. The five error categories are:

Category	Meaning
<code>value_substitution</code>	a different value was read back (e.g. FL240 → FL250)
<code>digit_transposition</code>	the same digits in a different order — a more specific, higher-risk classification of a value error (e.g. runway 21 → 12)
<code>omission</code>	an instructed item was not read back
<code>callsign_error</code>	callsign wrong or missing
<code>added_element</code>	the read-back contains an item not instructed

A key, defensible label-design choice is encoded here: which items are *mandatory* to read back versus informational. Wind, for instance, is informational, so a read-back that drops it is still a MATCH. The comparator is unit-tested (`tests/test_compare.py`) with no LLM or token required, and runs on every push via CI.

Two free backends. A single `Extractor` interface has two implementations selected by `EXTRACTOR_BACKEND : ollama` (local, default `qwen2.5:3b`) is used for development and the reproducible graded evaluation; `hf` (Hugging Face Inference API, `Qwen/Qwen2.5-7B-Instruct`) powers the hosted demo because the Streamlit Cloud free tier cannot run a local model. Both return the same typed fields, so the comparator and everything downstream are backend-agnostic and swapping is a one-line env change.

The Streamlit app. The user types the controller instruction and pilot read-back; the app runs the pipeline and returns MATCH, or DISCREPANCY with a human-readable list of specific problems and their categories. Input is text only.

4. Custom evaluation

Test set. We built our own labelled test set of **50 cases** (`eval/data/atc_readback_test_set.csv` , TC01–TC50), each an instruction/read-back pair with gold labels for `expected_verdict` , `error_category` , and `affected_field` . The set deliberately covers MATCH plus every error category, and includes hard cases by design (semantically-equal-but-differently-phrased read-backs, informational items like wind that must *not* trigger a discrepancy, and digit transpositions). Design and labelling are the team's own work.

Metric framing. The positive class is "the read-back contains an error" (a DISCREPANCY). Under that framing, precision/recall/F1 measure error **detection**. We additionally report the **false-alarm rate** — the fraction of genuinely correct read-backs that are wrongly flagged — which is the metric that

matters most in a safety setting, since false alarms erode controller trust. We also report verdict accuracy (MATCH vs DISCREPANCY), per-category detection recall, and a confusion matrix.

Headline result (`ollama` / `qwen2.5:3b` , 50 cases, final configuration):

Precision	Recall	F1	False-alarm rate	Verdict accuracy
0.971	1.000	0.986	6.2%	0.980

Confusion matrix (in prose). Of the 50 cases, 16 are gold MATCH and 34 are gold DISCREPANCY. The system correctly identifies 15 of the 16 correct read-backs (true negatives) and flags 1 correct read-back in error (one false alarm — the 6.2% false-alarm rate). It detects all 34 erroneous read-backs (34 true positives, zero false negatives), which is why recall is a perfect 1.000. The single overall verdict failure is that one false alarm.

Per-category detection recall. Detection recall is **100% in every error category**:

`value_substitution` (11/11), `omission` (8/8), `digit_transposition` (7/7), `callsign_error` (5/5), and `added_element` (3/3). Category accuracy is 0.980 and affected-field accuracy is 0.900. The full breakdown is in `eval/results/metrics.md` , regenerable with `make eval` .

5. Failure-mode analysis

We analysed failures across a **three-stage iteration** rather than as a single snapshot, because the iteration itself is the most informative result — it localises failure to the extraction step, not the comparator.

Stage	Extractor	F1	False-alarm	Verdict failures
Baseline	<code>qwen2.5:3b</code> , original prompt	0.957	12.5%	3
Hardened (regression)	<code>3b</code> , +rules +examples	0.944	25.0%	4
Final	<code>3b</code> , safe rules + comparator rule	0.986	6.2%	1

Stage 1 — baseline. Three distinct extraction weaknesses dominated: informational items leaking into clearance fields (the model mapped a *wind* "250 degrees / 8 knots" onto `heading=250` and `speed=8`), runway-side hallucination (inventing a "left" on one message and not the other), and inconsistent handling of multi-clause/gerund phrasing (missing a gerund-phrased "descending flight level 100").

Stage 2 — hardening regression and root cause. We hypothesised that adding explicit rules and more few-shot examples would suppress the hallucinations. It **regressed**: recall hit 100% but the false-alarm rate *doubled* from 12.5% to 25% (four false alarms — TCo8, TC11, TC12, TC13) and F1 fell.

Inspecting the extracted fields revealed why. An added example pairing a squawk with the `Easy 4471` callsign taught the 3B model to read the **4-digit callsign suffix as a squawk code** (`Easy 4471` → `squawk 4471`); this created one new false alarm (TC12) and mis-categorised two genuine discrepancies (TC37, TC50 — verdict still correct). Another example nudged it to misread `runway 24` as `heading 240` (TC13), and runway-side hallucination persisted (TCo8, TC11). The lesson is concrete and defensible: **over-specific few-shot examples can poison a small model** in ways that generalise badly.

Stage 3 — the principled fix. Two changes, neither a risky example: (1) a **deterministic comparator rule for runway side** (`_compare_runway`) that matches on the runway *number* and flags a side difference only when *both* messages explicitly state a side — model-independent and unit-tested; and (2) a clean prompt with **safe rules** (e.g. "squawk comes only from an explicit squawk instruction, never from the callsign's digits"). This resolved the wind leak (TC07), the runway-side false alarms (TC08, TC11), the squawk poisoning (fixing the TC37/TC50 mis-categorisations) and the heading hallucination (TC13), and caught the previously-missed added element (TC49), giving the best result on every metric. One false alarm remains (TC12), now for an unrelated reason discussed next.

The one remaining failure (TC12). On a two-clause instruction ("Easy 4471, reduce speed 210 knots, descend flight level 100." → "Speed 210 knots, descend flight level 100, Easy 4471."), the 3B model extracted `FL100` from the read-back but *dropped it from the instruction*, producing a spurious added-element discrepancy on a genuine MATCH. This is an extraction miss, not a comparator error — a capacity-bound limitation expected to resolve with a stronger extractor.

The same prompt and comparator were also run on `qwen2.5:7b` to test whether model capacity removes the residual extraction error. It does, decisively: 7B reaches **F1 1.000 with a 0% false-alarm rate and zero verdict failures**, correctly extracting the very two-clause instruction (TC12) the 3B model dropped. This confirms the residual failure is capacity-bound extraction, not a comparator flaw. The full 3B-vs-7B table and figures are in `notebooks/analysis.ipynb` (section 4). The overarching takeaways: the comparator is sound and extraction is the bottleneck; a reproducible harness turned "the prompt feels better" into a measured −18.8 pp false-alarm swing between attempts.

6. Limitations and future directions

This is a **text-only proof of concept**, not operational software; there is no auth, database, or live-radio audio path, and the hosted demo is best-effort while graded metrics come from the reproducible local run. Our own measurements show that **extraction is the bottleneck** — every observed failure traced to the LLM extraction step, never the deterministic logic. Small models also exhibit **variance and prompt sensitivity**: the same prompt can help or poison `qwen2.5:3b` depending on its examples, so results should be read as indicative of a 3B model under our prompt rather than a ceiling. The 50-case test set, while covering every category, is small, and a single false alarm or extraction miss moves the headline numbers noticeably. Future directions include an **ASR front end** (e.g. Whisper) to accept live radio audio, evaluation with **larger or fine-tuned extractor models** (the 3B-vs-7B comparison is a first step), a **larger and more diverse test set** with more accents and phraseology variants, phonetic/NATO-alphabet robustness, and confidence scoring with a controller-facing alerting UX.

7. Use of AI tools

Consistent with the assignment, we used an LLM assistant for brainstorming and scoping, boilerplate and scaffolding, first-draft prose, and AI-assisted literature search (every citation was checked against its primary source by a team member). All substantive intellectual decisions remained the team's own: the problem framing (verifying correctness, not transcribing), the test-set design and labels, the five-category error taxonomy, the comparison and normalization logic, the evaluation methodology including the false-alarm-rate and positive-class framing, and the interpretation of every result. Separately, the two Qwen2.5 extractor models are *part of the product*, not authoring aids, and perform

field extraction only. The full account is in [use_of_ai_tools.md](#), and every member is prepared to defend the design, labels, methodology, and results independently in the Q&A.

8. Conclusion

The ATC Readback Verifier demonstrates that read-back *verification* — distinct from transcription — can be performed with a transparent hybrid design in which a small free LLM extracts structured fields and deterministic Python makes every judgement. On our own 50-case test set the final configuration detects every erroneous read-back (recall 1.000), reaches F1 0.986, and keeps the safety-critical false-alarm rate to 6.2%, with 100% detection recall in all five error categories. The most valuable lesson is grounded directly in our measurements: across the three-stage iteration the comparator was never the cause of a failure, a naive prompt-hardening attempt *regressed* by poisoning the small model's extraction, and a principled fix combining a deterministic comparator rule with safe prompt rules recovered and improved the result. Extraction — not comparison — is the bottleneck, and the residual failure is capacity-bound, which is exactly what a stronger extractor or an ASR-driven extension would target next. As a course proof of concept the system is intentionally scoped, but it is reproducible, auditable, and honest about what it does and does not yet do.

References

Full reference list with verified source links is in [field_review.md](#). Citation numbers [1]–[19] in Section 2 above correspond one-to-one to that list.

Appendix A — Field Review (full, with references)

The ATC Readback Verifier operates at the intersection of aviation safety, spoken/written language understanding for air traffic control (ATC), and structured information extraction. In ATC, a controller issues an instruction (a clearance), the pilot repeats the safety-relevant elements back (the *read-back*), and the controller listens to confirm it is correct (the *hear-back*). This loop is the primary defence against miscommunication, yet it depends entirely on a human catching an error in real time. Our project targets the verification step of that loop for text input — extracting structured fields (callsign, altitude, heading, speed, frequency, squawk, runway, QNH) from both the instruction and the read-back, then comparing them deterministically and flagging discrepancies. Audio (ASR) is out of scope and treated as future work.

ATC communication & read-back safety

Read-back/hear-back failure is a long-recognised aviation safety problem. SKYbrary and the ICAO information paper on read-back and hear-back distinguish three failure modes: a read-back error the controller fails to catch, an absent read-back, and a "Type II" hear-back error where the controller does not notice a mistake in their *own* clearance even though the read-back was correct [1][2]. An uncorrected error can remain undetected until a loss of separation appears on the controller's display [1].

The error is quantified in foundational safety research. NASA's Aviation Safety Reporting System (ASRS) work by Monan coined the "hear-back problem," and an associated ASRS study analysed several hundred reports (one cited 417 reports over 29 months, April 1981–July 1983), concluding that a controller *not* challenging a read-back does not guarantee it is correct — crews must explicitly query doubtful clearances [3][4]. The FAA/DOT analysis of ASRS reports by Cardosi and colleagues classifies read-back/hear-back errors, absent read-backs, and Type II errors, and reports that controllers catch only roughly two-thirds of incorrect read-backs, leaving about a third undetected [5][6]. Volpe/FAA studies of TRACON voice communications report broadly consistent figures and find that error likelihood rises with message complexity (number of items per transmission), with headings, frequency changes, and altitudes among the most error-prone items [6]. (Specific TRACON percentages come from secondary summaries and should be confirmed against the primary PDF.) Consistent contributing factors across sources include similar/confusable callsigns, expectation bias (e.g. hearing heading 280 as flight level 280), high workload, frequency congestion, non-standard phraseology, and accent/language barriers [1][3]. ICAO Annex 10, PANS-ATM (Doc 4444), and the Manual of Radiotelephony (Doc 9432) define which safety-related items must be read back — callsign, runway, level/altitude, heading, speed, transponder code, QNH — which gives a verifier its ground-truth field set [7][8]. EUROCONTROL's European Action Plan for Air Ground Communications Safety reinforces strict read-back/hear-back discipline and standard phraseology [9].

Speech & language technology for ATC

A substantial body of speech and language work targets ATC. The ATCO2 corpus is the largest public resource: a ~4-hour gold test set with named-entity annotations and a ~5,281-hour pseudo-transcribed set, framing ATC understanding around three entity classes — callsign, command, and value — which map directly onto a read-back-verification pipeline [10][11]. ATCOSIM provides a smaller, cleaner ~10-hour benchmark of non-prompted controller speech with orthographic transcripts [12]. On modelling, transformer NER (fine-tuned BERT) is the standard baseline, with reported F1 around 0.97 for callsign, 0.82 for command, and 0.87 for value — command extraction being hardest because utterances chain

multiple commands and deviate from published ICAO phraseology [10]. Self-supervised models (wav2vec 2.0/XLS-R) and weakly-supervised Whisper have been fine-tuned for ATC and released openly, with Whisper-ATC checkpoints trained on ATCO2+ATCOSIM [13][14]. Fusing air-surveillance context (ADS-B/radar callsigns near the sector) substantially improves callsign recognition, the anchor for verifying a read-back targets the right aircraft [15].

Existing tools and the gap

Most mature and commercial ATC speech systems transcribe rather than verify. Appareo's Aviation Speech Recognition System converts ATC audio to text for cockpit displays but advertises no read-back or clearance-verification function [16]. The ATCO2 effort centres on collection, transcription, and concept extraction (callsign/command/value) — the building blocks for verification, but not a fielded verifier [10][11]. MITRE/MIT prototypes likewise focus on transcription plus information extraction for conformance monitoring [17]. Dedicated read-back-error-detection (RED) prototypes do exist, almost entirely from European SESAR-funded research: Helmke and colleagues in the HAAWAI project explicitly perform verification — audio → text → concept, then comparing controller-instruction concepts against pilot read-back concepts using rule-based and data-driven methods — reporting roughly 80–81% detection on real recordings, with detection at the semantic "concept" level proving more robust than word-level matching [18][19]. The gap is therefore clear: verification systems remain largely at the lab/prototype stage with non-trivial false-alarm rates, and no widely deployed tool performs full automated pilot-read-back-vs-controller-instruction verification, even though ICAO doctrine requires the check but leaves it a manual human task [2][6]. Our contribution is a lightweight, auditable, text-based field-by-field verifier with an explicit error taxonomy, positioned squarely in this under-served niche.

How our approach fits

Our design mirrors the HAAWAI concept-comparison insight while staying transparent: an LLM performs the extraction step — turning free-text instructions and read-backs into the same structured fields used as ICAO mandatory read-back items [7][18] — and every correctness judgement is made by deterministic Python, not by the model. This keeps the verification logic auditable and reproducible, separating fallible language understanding from the comparison that actually flags discrepancies, and scopes us to text so the contribution is a focused verification layer rather than another transcription front-end.

References

1. Read-back or Hear-back — SKYbrary Aviation Safety. Defines read-back/hear-back, the three error types, mandatory read-back items, and causal factors. <https://skybrary.aero/articles/read-back-or-hear-back>
2. Importance of ATC Readback and Hearback (ATM/SG/13 IP/o6, 2025) — ICAO APAC. Primary ICAO information paper confirming definitions, Doc 4444/Annex 10 mandatory items, and contributing factors. <https://www.icao.int/sites/default/files/APAC/Meetings/2025/2025%20ATMSG13/04-Information%20Papers/IPo6%20Importance%20of%20ATC%20Readback%20and%20Hearback%20%20.pdf>
3. Readback/Hearback (ASRS Directline) — Bill Monan, NASA ASRS. Foundational analysis coining the "hear-back problem"; lists major causal patterns. https://asrs.arc.nasa.gov/publications/directline/dl1_read.htm

4. Human Factors in Aviation Operations: The Hearback Problem — NASA Technical Reports Server. The 417-reports / 29-month ASRS hear-back study and three-type error taxonomy. <https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/19890007398.pdf>
5. Pilot-Controller Communication Errors: An Analysis of ASRS Reports (Cardosi, Falzarano) — Semantic Scholar / DTIC ADA356655. Volpe/FAA study; controllers catch only ~66% of incorrect read-backs. <https://www.semanticscholar.org/paper/Pilot-Controller-Communication-Errors:-An-Analysis-Cardosi-Falzarano/0aa6945d2387486c00c26b6d115b20a94fe6b38b>
6. An Analysis of TRACON Controller-Pilot Voice Communications — Volpe/FAA (ROSAP/BTS). Primary report behind the ~1% read-back-error / ~40%-uncaught / 13,089-transmission figures (confirm in PDF). https://rosap.ntl.bts.gov/view/dot/8711/dot_8711_DS1.pdf
7. Manual of Radiotelephony (Doc 9432, 4th ed. 2007) — ICAO Store. Official phraseology manual; standard-phraseology basis a verifier should normalize against. <https://store.icao.int/en/manual-of-radiotelephony-doc-9432>
8. Standard Phraseology — SKYbrary Aviation Safety. Explains that standard phraseology exists to make read-back/hear-back errors easier to detect. <https://skybrary.aero/articles/standard-phraseology>
9. European Action Plan for Air Ground Communications Safety (AGC) — EUROCONTROL. Recommends strict read-back/hear-back procedure and standard phraseology. <https://www.eurocontrol.int/publication/european-action-plan-air-ground-communications-safety>
10. ATCO2 corpus: A Large-Scale Dataset for ASR and NLU of ATC Communications (Zuluaga-Gomez et al., arXiv:2211.04054). Defines callsign/command/value NER, dataset sizes, and BERT baseline F1 scores. <https://arxiv.org/abs/2211.04054>
11. Lessons Learned in Transcribing 5000h of ATC Communications (MDPI Aerospace 2023 / arXiv:2305.01155). Documents the ATCO2 transcription/annotation pipeline and ICAO phraseology deviation challenges. <https://www.mdpi.com/2226-4310/10/10/898>
12. The ATCOSIM Corpus of Non-Prompted Clean Air Traffic Control Speech (Hofbauer et al., LREC 2008). ~10h clean simulation ATC speech, TU Graz + Eurocontrol EEC, freely available. <https://aclanthology.org/L08-1507/>
13. How Does Pre-trained Wav2Vec 2.0 Perform on Domain-Shifted ASR? (Zuluaga-Gomez et al., arXiv:2203.16822, IEEE SLT). Benchmarks wav2vec2/XLS-R on ATC; ~20–40% relative WER reduction vs hybrid baselines. <https://arxiv.org/abs/2203.16822>
14. WhisperATC: Applying Large-Scale Weakly-Supervised ASR to Air Traffic Control (GitHub). Open-source fine-tuning of Whisper on ATCO2/ATCOSIM (verify exact WER claims in the paper). <https://github.com/jlvdoorn/WhisperATC>
15. Improving callsign recognition with air-surveillance data in air-traffic communication (Nigmatulina, Zuluaga-Gomez et al., arXiv:2108.12156, Interspeech 2021). Surveillance-context boosting; large callsign-accuracy/WER gains. <https://arxiv.org/pdf/2108.12156>
16. Aviation Speech Recognition System (ATC Transcription) — Appareo. Commercial product that transcribes ATC audio but advertises no read-back/clearance verification. <https://appareo.com/aviation/aviation-speech-recognition-system/>
17. Automatic Speech Recognition for Air Traffic Control Communications (Badrinath & Balakrishnan, MIT/MITRE, TRR 2021). ATC-tuned ASR transcribing voice and extracting callsign/runway for conformance monitoring. <https://dspace.mit.edu/handle/1721.1/145275>
18. Readback Error Detection by Automatic Speech Recognition to Increase ATM Safety (Helmke et al., HAAWAI, ATM Seminar 2021). Rule-based + data-driven RED via concept comparison; ~80–81%

detection at low false-alarm rate. https://publications.idiap.ch/attachments/papers/2021/Helmke_ATMSEMINAR-2_2021.pdf

19. Better automatic speech recognition for safer air traffic control (HAAWAI, CORDIS, European Commission). >95% controller / >90% pilot word recognition, ~97% callsign with radar fusion, ~80% RED at low false-alarm rate (lab evaluation). <https://cordis.europa.eu/article/id/442201-better-automatic-speech-recognition-for-safer-air-traffic-control>

Appendix B — Failure-Mode Analysis (full)

All numbers come from the committed runs in `eval/results/runs/` and are reproduced by `notebooks/analysis.ipynb`.

We analysed failures not as a one-off snapshot but across a **three-stage iteration**, because the iteration itself is the most informative result: it shows *where* the system fails (the extraction step, not the comparator) and how different fixes help or hurt.

Summary of the iteration

Stage	Extractor	Precision	Recall	F1	False-alarm	Verdict acc	Verdict failures
Baseline	qwen2.5:3b, original prompt	0.943	0.971	0.957	12.5%	94.0%	3
Hardened (regression)	qwen2.5:3b, +rules +examples	0.895	1.000	0.944	25.0%	92.0%	4
Final	qwen2.5:3b, safe rules + comparator rule	0.971	1.000	0.986	6.2%	98.0%	1

The deterministic comparator is unit-tested and was **never** the cause of a failure; every error below is an **extraction** error or a deliberate **label** decision. The 3B-vs-7B comparison (same prompt) is in the notebook.

Stage 1 — baseline failures (qwen2.5:3b, original prompt)

Case	Instruction → read-back	Gold	Predicted	Cause (hypothesis)
TC07	"...cleared to land runway 24, wind 250 degrees 8 knots. "	MATCH	DISCREPANCY	Model mapped the wind "250 degrees / 8 knots" onto <code>heading=250</code> and <code>speed=8</code> → false alarm. Informational items leaking into clearance fields.
TC11	"...line up and wait runway 18. " (no side)	MATCH	DISCREPANCY	Model hallucinated a runway side ("left" on one message, none on the other) → false alarm on a non-existent side difference.
TC49	read-back adds " descending flight level 100 " not instructed	DISCREPANCY (added_element)	MATCH	Model failed to extract the gerund-phrased altitude from the read-back → missed a real added element.
TC28	squawk 5701 → 5071	DISCREPANCY / transposition	DISCREPANCY / substitution	Model misread the read-back squawk digits (5071 → 0507), so the digit-multiset no longer matched → mis-categorised (verdict still correct).
TC39	"cleared to land runway 24" → "Cleared to land" (no runway)	DISCREPANCY / omission	DISCREPANCY / substitution	Model hallucinated runway 24 into the read-back, turning an omission into a value error (verdict still correct).

Read-out: three distinct extraction weaknesses dominate — (a) informational items (wind) leaking into fields, (b) **runway-side hallucination**, and (c) inconsistent handling of multi-clause / gerund phrasing.

Stage 2 — a prompt-hardening attempt that backfired

Hypothesis: adding explicit rules and more few-shot examples would suppress the hallucinations and cut the false-alarm rate. **Result: it regressed** — recall reached 100% (it now caught the previously-missed TC49) but the false-alarm rate *doubled* from 12.5% to 25% and F1 fell. The hardened run's **four false alarms were TCo8, TC11, TC12, TC13** — all genuine MATCHes wrongly flagged. Inspecting the extracted fields showed two new poisoning effects from the added examples:

- **Squawk poisoning (Easy 4471 → squawk 4471).** An added example pairing a squawk with the Easy 4471 callsign taught the 3B model to read the **4-digit callsign suffix as a squawk code**. This produced **one new false alarm (TC12)** and additionally **mis-categorised two genuine discrepancies**: TC37 (a squawk *omission*) and TC50 (an *added* squawk) were both relabelled `value_substitution` — the verdict stayed correct, but category accuracy fell.
- **TC13 — heading hallucination.** An added example pairing a heading with a "descending" altitude nudged the model to misread `runway 24` as `heading 240`, inventing a heading in the read-back.

- **Runway-side hallucination got worse, not better:** the hardened run still invented a side on both TCo8 and TC11, so the extra examples did not address it.

Lesson: over-specific few-shot examples can *poison* a small model in ways that generalise badly. This is a concrete, defensible finding about prompt engineering with small models.

Stage 3 — the principled fix

Two changes, neither of which is a risky example:

1. **Deterministic comparator rule for runway side** (`src/atc_verifier/compare.py` , `_compare_runway`): match on the runway **number**; flag a *side* difference only when **both** messages explicitly state a side. Rationale: an unstated side is not a read-back error, and the model hallucinates sides. This is model-independent and unit-tested.
2. **Clean prompt with safe rules:** e.g. "squawk comes only from an explicit squawk instruction; never from the callsign's digits", and an explicit instruction-form wind example — **rules**, not poisoning examples.

This combination resolved the wind leak (TCo7), the runway-side false alarms (TCo8, TC11), and the squawk poisoning — TC37 and TC50 are now categorised correctly and the heading hallucination (TC13) is gone — and it caught the previously-missed added element (TC49), giving the best result on **every** metric: F1 0.986, false-alarm 6.2%, recall 1.000, and **100% detection recall in all five categories**. A single false alarm remains (TC12), now for an unrelated reason — see below.

Remaining failure (1 case)

Case	Instruction → read-back	Gold	Predicted	Cause
TC12	"Easy 4471, reduce speed 210 knots, descend flight level 100. " → "Speed 210 knots, descend flight level 100, Easy 4471."	MATCH	DISCREPANCY (added_element, altitude)	The 3B model extracted <code>FL100</code> from the read-back but dropped it from the instruction — inconsistent extraction on a two-clause instruction — producing a spurious added element. A genuine small-model limitation; likely resolved by a stronger extractor (see the 3B-vs-7B comparison).

3B vs 7B

The same prompt and comparator were run with `qwen2.5:7b` to test whether model capacity resolves the residual extraction error. It does — and decisively:

Model	Precision	Recall	F1	False-alarm	Verdict accuracy	Verdict failures
<code>qwen2.5:3b</code>	0.971	1.000	0.986	6.2%	0.980	1 (TC12)
<code>qwen2.5:7b</code>	1.000	1.000	1.000	0.0%	1.000	0

The 7B model produces a **perfect verdict score** on the test set: zero false alarms, zero missed errors. Critically, it correctly extracts the two-clause instruction in **TC12** that the 3B model dropped — confirming directly that the residual 3B failure is **capacity-bound extraction**, not a flaw in the

(identical) comparator. The only blemish for 7B is a single *category* mismatch (TC20: the 7B model failed to extract the read-back frequency, so a frequency value-substitution surfaces as an `omission`) — the verdict is still correct (DISCREPANCY). The comparison table and chart are reproduced in `notebooks/analysis.ipynb` (section 4); raw outputs are in `eval/results/runs/7b_final_v3/`.

Takeaways

1. **The comparator is sound; extraction is the bottleneck.** Every failure traced to the LLM extraction step, never the deterministic logic.
2. **Few-shot examples can backfire** on small models — safe rules and deterministic post-processing were more reliable levers than adding examples.
3. **Measuring made it possible.** A reproducible harness turned "the prompt feels better" into a measured −18.8 pp false-alarm swing between attempts.
4. **Residual errors are capacity-bound** — multi-clause extraction misses are the kind of thing a larger model is expected to fix, which the 3B-vs-7B comparison tests directly.

Appendix C — Use of AI Tools (full)

The assignment explicitly encourages using LLMs, while requiring that the **substantive intellectual decisions remain the team's own**. This section records how we used AI assistance and where the team's own judgement governed.

Where we used AI assistance

- **Brainstorming and scoping.** We used an LLM assistant to pressure-test the project shape (hybrid LLM-extraction + deterministic comparison), the choice of free/local tooling, and the repository structure.
- **Boilerplate and scaffolding.** Packaging (`requirements` , `pyproject` , `Makefile`), the Streamlit app shell and styling, the CI workflow, the evaluation harness plumbing, and documentation drafts were AI-assisted, then reviewed by us.
- **Prompt drafting and iteration.** The few-shot extraction prompt was drafted with AI help; we then iterated it against our own failure cases (see the failure analysis) and decided which fixes to keep based on measured results.
- **Drafting prose.** First drafts of this report's sections, the README, the install guide, and the user manual were AI-assisted. The team edited and verified them.
- **Literature search.** We used AI-assisted web search to locate candidate sources for the field review; **every citation was checked against the original source by a team member** before inclusion.

What the team decided and owns

These were **not** delegated to the model; they are our own decisions and we can defend each one:

- **Problem framing** — verifying read-back *correctness*, not transcription.
- **Test-set design and labels** — the 50 instruction/read-back pairs and their `expected_verdict` / `error_category` / `affected_field` labels, including the deliberate hard cases.
- **The error taxonomy** — the five categories (`value_substitution` , `digit_transposition` , `omission` , `callsign_error` , `added_element`) and their definitions.
- **Comparison logic** — what counts as a match, the normalization rules (units, spelled-out numbers, telephony, trailing zeros), and the policy on which items are *mandatory* to read back (e.g. wind is informational and not required).
- **Evaluation methodology** — the metric choices, including reporting false-alarm rate, and the framing of error detection as the positive class.
- **Error interpretation** — the hypotheses in the failure analysis and the conclusions drawn from the metrics.

Tools used

Tool	Used for
LLM coding/writing assistant	brainstorming, scaffolding, draft code & prose, debugging, literature search

Two LLMs are **part of the product itself**, not authoring aids: the extractor backends `qwen2.5:3b` (local, via Ollama) and `Qwen/Qwen2.5-7B-Instruct` (Hugging Face Inference API). These perform **field extraction only** and are a component of the system, documented in the system design.

Verification & accountability

Code produced with assistance is covered by unit tests (`tests/test_compare.py`) and a reproducible evaluation run; prose was fact-checked against the actual repository; citations were verified against primary sources. Where AI output was wrong or unsuitable, we corrected or discarded it.

Q&A reminder: every member must be able to **explain and defend** the design, label definitions, methodology, and results, independent of any AI assistance used to produce them.